TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN

GROUP REPORT

COURSE: DATA STRUCTURES AND ALGORITHMS

PROJECT: STRING COMPRESSION

Academic Advisor: Trần Hoàng Quân

Presented by / Students:

Châu Tiến Phát - 25127452

Nguyễn Quốc Khánh - 25127076

Nguyễn Song Nhật Tiến - 25127519

Nguyễn Anh Quang Tiến - 25127518

HO CHI MINH, 2026

CONTENTS

Section 1. Group Information

Student Name	Student ID	Assigned Tasks	Completed
Châu Tiến Phát **(Leader)**	25127452	- Core CLI framework & tool architecture (main.cpp, ICompressor interface, readme.txt). - LZW algorithm implementation. - Report: Problem Statement (Section 2) & LZW analysis (Section 3.3). - Demo video recording.	100%
Nguyễn Quốc Khánh	25127076	- QA / testing (round-trip verification across all algorithms). - Review source code, reconstruct project folder arrangement. - Report: Experimental Results (Section 4) & References (Section 5). - Video recording, upload & video.txt. - Final submission packaging (zip, Report.pdf export).	100%
Nguyễn Song Nhật Tiến	25127519	- Huffman Coding implementation. - Report: Huffman analysis (Section 3.2). - Deflate algorithm implementation (bonus). - Report: Deflate analysis (Section 7.6–7.11).	100%
		- Run-Length Encoding	

Nguyễn Anh Quang Tiến	25127518	(RLE) implementation. - Video editing, aftereffect. - LZ77 algorithm implementation (bonus). - Report: RLE analysis (Section 3.1) & LZ77 analysis (Section 7.1–7.5). - Test data generation (scenario1/scenario2 scripts).	100%
		Total Group Completion	100%

Section 2 (Problem Statement)

Formal Definition of Lossless Data Compression

Lossless data compression is the class of data encoding algorithms that reduce the storage size of a data object while guaranteeing that the original data can be perfectly reconstructed from its compressed representation. Formally, given an input byte sequence $X = x_1 x_2 ... x_N$ of length N bytes, a lossless compression scheme consists of two functions: Encode: C = Compress(X), where C is the compressed byte sequence of length $M \leq N$ (ideally $M \ll N$); and Decode: X' = Decompress(C), such that X' = X exactly (bit-for-bit identical).

The quality of a compression algorithm is evaluated by two primary metrics: Compression Ratio = Original Size (N) / Compressed Size (M), where a ratio greater than 1 indicates effective compression; and Space Savings = 1 − (M / N), expressed as a percentage, where positive values indicate the fraction of storage saved.

Different algorithms exploit different structural properties of the input data: statistical redundancy (Huffman Coding), sequential redundancy (Run-Length Encoding), and substring repetition (LZW, LZ77). No single algorithm is universally optimal; each has scenarios where it excels or degrades.

Running Example

Throughout Section 3, all algorithms are demonstrated using the same running example string to enable direct comparison of their behavior and output: BABBACAC.

This string has 8 characters (N = 8 bytes in ASCII) drawn from an alphabet of K = 3 unique symbols: {A, B, C}. The character frequency distribution is:

Character	Frequency	Relative Frequency
B	3	3/8 = 37.5%
A	3	3/8 = 37.5%
C	2	2/8 = 25.0%

This string was chosen because it stress-tests different algorithms at once: no long runs of identical characters (challenging for RLE), a non-uniform but not extreme frequency distribution (moderate gain for Huffman), and repeated multi-character substrings like “BA” and “AC” (advantageous for LZW).

Section 3.1 (Algorithm Analysis - Run-Length Encoding)

3.1.1 Introduction & Application

- **Core Idea:** Run-Length Encoding (RLE) is a foundational, lossless byte-level data compression algorithm. It operates by scanning a sequence of data and replacing consecutive repeating occurrences of identical bytes (a "run") with a single instance of the byte value alongside a count byte representing the run length.
- **Real-world Applications:** RLE is widely utilized in palette-based bitmap images (e.g., BMP, PCX, Targa), black-and-white fax transmission protocols (ITU-T T.30 standard), and as an intermediate compression stage in formats like JPEG and H.264.

3.1.2 Pseudocode

- **Compression:**

```
Algorithm RLECompress(inputFile, outputFile):
    Input: Path to original file
    Output: Binary compressed file with byte-count pairs
    buffer <- ReadAllBytes(inputFile)
    if buffer.IsEmpty() then:
        CreateEmptyFile(outputFile)
        return

    n <- buffer.Size()
    i <- 0
    writer <- OpenBinaryWriter(outputFile)
    while i < n do:
        currentChar <- buffer[i]
        count <- 1
        while (i + 1 < n) and (buffer[i + 1] == currentChar) and (count < 255)
do:
            count <- count + 1
            i <- i + 1
        writer.WriteByte(currentChar)
        writer.WriteByte(count)
        i <- i + 1
    CloseFiles()
```

- **Decompression:**

```
Algorithm RLEDecompress(inputFile, outputFile):
    Input: Binary compressed file containing byte-count pairs
    Output: Reconstructed original file
```

```
reader <- OpenBinaryReader(inputFile)
writer <- OpenBinaryWriter(outputFile)
while not reader.IsEOF() do:
    currentChar <- reader.ReadByte()
    count <- reader.ReadByte()
    for k <- 1 to count do:
        writer.WriteByte(currentChar)
CloseFiles()
```

3.1.3 Step-by-Step Trace & Output Structure

- **Input String:** BABBACAC
- **Total Original Size (N):** 8 bytes (Uncompressed ASCII)

Step-by-Step Execution Trace:

Step	Current Index	Current Byte	Run Count	Pair Output (Byte, Count)	Cumulative Output Size
1	Index 0	'A'	1	('B', 1)	2 bytes
2	Index 1	'A'	1	('A', 1)	4 bytes
3	Index 2–3	'B'	2	('B', 2)	6 bytes
4	Index 4	'A'	1	('A', 1)	8 bytes
5	Index 5	'C'	1	('C', 1)	10 bytes
6	Index 6	'A'	1	('A', 1)	12 bytes

7	Index 7	'C'	1	('C', 1)	14 bytes

- **Encoded Stream:** `(B, 1), (A, 1), (B, 2), (A, 1), (C, 1), (A, 1), (C, 1)`
- **Result:** Total output size = 14 bytes (compared to 8 bytes original ASCII).

3.1.4 Complexity Analysis (Big-O)

Phase	**Time Complexity**	**Space Complexity**	**Explanation**
Read, Buffer Load	***O***(N)	***O***(N)	Linear pass reading input bytes into memory buffer.
Run-Length Encoding	***O***(N)	***O***(1)	Single pass traversing the array, inner count increment runs linearly.
Stream Output	***O***(N)	***O***(1)	Direct byte writing proportional to output pairs.
Decompression	***O***(N)	***O***(1)	Recreating N original bytes by iterating through byte-count pairs.
Total Complexity	***O***(N)	***O***(1)	Asymptotic space complexity excluding required storage buffers.

3.1.5 Discussion: Performance Boundaries & Limitations

1. **Best-Case Scenario**
    - **Data Characteristic:** Long contiguous sequences of identical characters (e.g. AAAAA...).
    - **Effect:** Maximum compression efficiency. A run of 255 identical bytes compresses into 2 bytes, yielding a maximum compression ratio of approximately 127.5 times.
2. **Worst-Case Scenario**
    - **Data Characteristic:** High-entropy or alternating data with no contiguous duplicates (e.g., BABBACAC or ABCDEF...).
    - **Effect:** Every single byte requires an additional count byte, resulting in a **100% file size expansion** (2N bytes output) and negative space savings.
3. **Run-Count Overflow Boundary**
    - Since run counts are stored as an 8-bit unsigned integer (uint8_t, max value 255), continuous streams exceeding 255 identical bytes are split across multiple pairs (byte, 255), adding 2 extra bytes per 255-byte block.

Section 3.2 (Algorithm Analysis - Huffman)

3.2.1 Introduction & Application

- **Core Idea:** Huffman Coding is a classic greedy algorithm used for lossless data compression. It assigns variable-length bit codes to input characters based on their frequencies: characters that appear more frequently are assigned shorter binary codes, while less frequent characters receive longer codes. It guarantees optimal prefix codes, meaning no code is a prefix of another, eliminating ambiguity during decoding.
- **Real-world Applications:** Huffman Coding is widely used as the final entropy encoding step in standard file formats such as **JPEG images**, **MP3 audio**, and **DEFLATE/ZIP archives**.

3.2.2 Pseudocode

- **Compression**:

```
Algorithm HuffmanCompress(inputFile, outputFile):
        Input: Path to original file
        Output: Binary compressed file containing header and bitstream
          freqMap <- CountFrequency(inputFile)
          minHeap <- InitializeEmptyMinHeap()
          for each (symbol, count) in freqMap do:
              node <- CreateLeafNode(symbol, count)
              minHeap.Insert(node)
          while minHeap.Size() > 1 do:
              left <- minHeap.ExtractMin()
```

```
        right <- minHeap.ExtractMin()
        parent <- CreateInternalNode(freq = left.freq + right.freq)
        parent.left <- left
        parent.right <- right
        minHeap.Insert(parent)
    root <- minHeap.ExtractMin()
    codeTable <- InitializeEmptyTable()
    GenerateCodes(root, currentCode = "", codeTable)
    WriteHeader(outputFile, totalBytes, freqMap)
    bitWriter <- OpenBitWriter(outputFile)
    for each byte in ReadBytes(inputFile) do:
        code <- codeTable[byte]
        bitWriter.WriteBits(code)
    bitWriter.FlushBuffer()
    CloseFiles()
```

- **Decompression:**

```
Algorithm HuffmanDecompress(inputFile, outputFile):
    Input: Binary compressed file
    Output: Reconstructed original text file
    (totalBytes, freqMap) <- ReadHeader(inputFile)
    if totalBytes == 0 then:
        CreateEmptyFile(outputFile)
        return
    minHeap <- InitializeEmptyMinHeap()
    for each (symbol, count) in freqMap do:
        node <- CreateLeafNode(symbol, count)
        minHeap.Insert(node)

    while minHeap.Size() > 1 do:
        left <- minHeap.ExtractMin()
        right <- minHeap.ExtractMin()
        parent <- CreateInternalNode(freq = left.freq + right.freq)
        parent.left <- left
        parent.right <- right
        minHeap.Insert(parent)

    root <- minHeap.ExtractMin()
    bitReader <- OpenBitReader(inputFile)
    currentNode <- root
    decodedCount <- 0
    while decodedCount < totalBytes do:
```

```
            bit <- bitReader.ReadBit()
            if bit == 0 then:
                currentNode <- currentNode.left
            else:
                currentNode <- currentNode.right
            if IsLeaf(currentNode) then:
                WriteByte(outputFile, currentNode.symbol)
                decodedCount <- decodedCount + 1
                currentNode <- root
        CloseFiles()
```

3.2.3 Step-by-Step Trace & Binary Tree Visualization

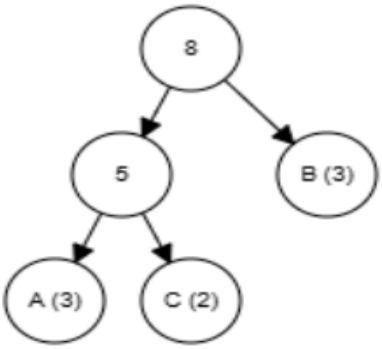
Step 1: Character Frequency Table

- Total characters (N): 8
- Unique characters (K): 3
  - B: 3
  - A: 3
  - C: 2

Step 2: Min-Heap Construction & Merging Steps

1. **Initial Min-Heap state:** [(C, 2), (A, 3), (B, 3)]
2. **Iteration 1:**
   - Extract two smallest nodes: C (freq: 2) and A (freq: 3).
   - Create parent node Internal_1 (freq: 2 + 3 = 5).
   - Insert Internal_1 into Min-Heap.
   - **Min-Heap state:** [(B, 3), (Internal_1, 5)]
3. **Iteration 2:**
   - Extract two smallest nodes: B (freq: 3) and Internal_1 (freq: 5).
   - Create root node Root (freq: 3 + 5 = 8).
   - Insert Root into Min-Heap.
   - **Min-Heap state:** [(Root, 8)] (Tree construction complete).

Step 3: Complete Huffman Binary Tree Structure



Generated Prefix Codes:

- B → 0 (1 bit)
- C → 10 (2 bits)
- A → 11 (2 bits)

Step 4: Encoded Bitstream

- Input string: B A B B A C A C
- Bitstream: 0 11 0 0 11 10 11 10
- **Result:** 0110011101110 (Total: 13 bits, compared to 64 bits uncompressed ASCII).

3.2.4 Complexity Analysis (Big-O)

Phase	Time Complexity	Space Complexity	Explanation
Frequency Counting	***O***(N)	***O***(K)	Single pass reading through the entire input stream.
Min-Heap Initialization	***O***(K)	***O***(K)	Inserting ***K*** leaf nodes into the Min-Heap.
Tree Construction	***O***(K log K)	***O***(K)	Performs ***K-1*** iterations; each iteration involves 2 **pop** and 1 **push** operations taking ***O(log K)*** time.
Code Generation	***O***(K)	***O***(K)	Recursive tree traversal (DFS) visiting all ***2K - 1*** nodes.
Encoding Process	***O***(N)	***O***(1)	Re-reading the input stream and doing ***O***(1) code lookup per byte.
Total Complexity	***O*(N + K log K)**	***O*(K)**	***Overall asymptotic complexity.***

3.2.5 Discussion: Performance Boundaries & Limitations

1. Best-Case Scenario

- **Data Characteristic:** Skewed character distribution with highly unequal frequencies (e.g., exponential distribution ***1, 2, 4, 8,.....***).
- **Effect:** Yields a deeply asymmetric, unbalanced Huffman tree. The most dominant character receives a concise 1-bit code, yielding maximum compression ratio.

2. Worst-Case Scenario

- **Data Characteristic:** Uniform character distribution (all characters appear with nearly identical frequency, such as encrypted or randomized files).
- **Effect:** Results in a completely balanced binary tree where every character is assigned a code length of approximately ≈ $\log_2$**K** bits (8 bits for **K**=256). Compression ratio approaches **1.0x** (no gain).

3. Small File Overhead (Negative Compression)

For small input files (**< 100** bytes), the fixed metadata header stored in the output file (which includes total byte count N, alphabet size K, and frequency pairs) can exceed the space saved by bit-level encoding. This leads to a **negative space savings percentage**

(Compressed Size > Original Size).

Section 3.3 (Algorithm Analysis - LZW)

3.3.1 Introduction & Application

Core Idea: The Lempel-Ziv-Welch (LZW) algorithm is a universal, dictionary-based lossless data compression algorithm. Unlike statistical methods such as Huffman Coding that require a preliminary frequency analysis pass, LZW builds its dictionary adaptively in a single pass over the input data. It starts with a base dictionary containing all 256 single-byte entries (codes 0–255) and dynamically learns new multi-character substrings as they are encountered. Each time a previously unseen substring is found, it is added to the dictionary with the next available code, and the longest known prefix is emitted as a variable-width integer code. The decoder reconstructs the same dictionary in lockstep, so no dictionary metadata needs to be stored in the compressed file — making LZW self-contained.

Real-world Applications: LZW is the core compression engine behind the GIF image format (12-bit variable-width LZW), the Unix compress utility (adaptive code widths up to 16 bits), the TIFF image format (LZW compression option), and the V.42bis modem compression standard. Its single-pass, adaptive nature makes it well-suited for streaming data where a second pass is impractical.

3.3.2 Pseudocode

- **Compression:**

```
Algorithm LZWCompress(inputFile, outputFile):
    Input: Path to original file
    Output: Binary file containing variable-width integer codes

    dictionary <- InitSingleByteEntries()   // codes 0..255
    nextCode <- 256
    codeWidth <- 9              // starts at 9 bits
    maxCodeWidth <- 16
    dictLimit <- 2^maxCodeWidth // 65536
    w <- ""                     // current longest match
    writer <- OpenBitWriter(outputFile)

    for each byte c in inputFile do:
        wc <- w + c
        if wc in dictionary then:
            w <- wc              // extend the match
        else:
            writer.WriteBits(dictionary[w], codeWidth)
            if nextCode < dictLimit then:
                dictionary[wc] <- nextCode
                nextCode <- nextCode + 1
                if nextCode == 2^codeWidth and codeWidth < maxCodeWidth then:
                    codeWidth <- codeWidth + 1
            w <- c               // reset to single char

    if w != "" then:
        writer.WriteBits(dictionary[w], codeWidth)
    writer.Flush()
    CloseFiles()
```

- **Decompression:**

```
Algorithm LZWDecompress(inputFile, outputFile):
    Input: Compressed binary file containing variable-width codes
    Output: Reconstructed original file

    dictionary <- InitSingleByteEntries()   // codes 0..255
    nextCode <- 256
    codeWidth <- 9
    maxCodeWidth <- 16
```

```
dictLimit <- 2^maxCodeWidth
reader <- OpenBitReader(inputFile)
code <- reader.ReadBits(codeWidth)
prev <- dictionary[code]
Write(outputFile, prev)

while reader.ReadBits(codeWidth) -> code do:
    if code < |dictionary| then:
        entry <- dictionary[code]
    else if code == |dictionary| then:
        entry <- prev + prev[0]   // special "cScSc" case
    else:
        Error("Corrupted stream")
    Write(outputFile, entry)
    if nextCode < dictLimit then:
        dictionary[nextCode] <- prev + entry[0]
        nextCode <- nextCode + 1
        // Decoder widens one step earlier than encoder
        if nextCode == 2^codeWidth - 1 and codeWidth < maxCodeWidth then:
            codeWidth <- codeWidth + 1
    prev <- entry
CloseFiles()
```

3.3.3 Step-by-Step Trace

Using the running example string BABBACAC (8 bytes, alphabet {A, B, C}), we trace the LZW compression algorithm. The dictionary is initialized with 256 single-byte entries (codes 0–255); for clarity, only the relevant entries are shown.

Initial Dictionary (relevant entries):

Code	Entry
65	"A"
66	"B"
67	"C"
...	(all other single bytes)

Compression Trace:

Step	c	w	wc	In Dict?	Action	Code	New Entry
1	B	""	"B"	Yes	Extend: w ← "B"	—	—
2	A	"B"	"B A"	No	Emit code for "B"; learn "BA"; reset w←"A"	66	256 → "BA"
3	B	"A"	"A B"	No	Emit code for "A"; learn "AB"; reset w←"B"	65	257 → "AB"
4	B	"B"	"B B"	No	Emit code for "B"; learn "BB"; reset w←"B"	66	258 → "BB"
5	A	"B"	"B A"	Yes ✓	Extend: w ← "BA" (code 256 matches!)	—	—
6	C	"B A"	"B AC"	No	Emit code for "BA"; learn "BAC"; reset w←"C"	256	259 → "BAC"
7	A	"C"	"C A"	No	Emit code for "C"; learn "CA"; reset w←"A"	67	260 → "CA"
8	C	"A"	"A C"	No	Emit code for "A"; learn "AC"; reset w←"C"	65	261 → "AC"
End	—	"C"	—	—	Emit final code for "C"	67	—

Summary of Encoded Output: The encoder emits the code sequence 66, 65, 66, 256, 67, 65, 67 — that is, 7 codes at 9 bits each = 63 bits = 8 bytes (with 1 padding bit), compared to the original 8 bytes (64 bits). On this tiny input, LZW achieves approximately 1:1 compression (verified directly against the compiled program's output: 8 bytes in, 8 bytes out). However, the dictionary learned 6 new multi-character substrings that would yield significant savings on longer, repetitive inputs.

Final Dictionary State:

Code	Entry	Learned At Step
256	"BA"	Step 2
257	"AB"	Step 3
258	"BB"	Step 4
259	"BAC"	Step 6
260	"CA"	Step 7
261	"AC"	Step 8

Key observation at Step 5: the substring “BA” was learned at Step 2 (code 256), and reused at Step 5 when the encoder matched a 2-character substring in a single dictionary lookup. This demonstrates LZW's adaptive learning — the longer the input, the more multi-character entries accumulate, and the more bytes each emitted code represents.

3.3.4 Complexity Analysis (Big-O)

Let N be the input length in bytes and K be the maximum dictionary size (in this implementation, $K = 2^{16} = 65{,}536$).

Phase	Time	Space	Explanation
Dictionary Init	***O***(256) = O(1)	***O***(256) = ***O***(1)	Insert 256 single-byte entries into the hash map. Constant cost.
Compression (main loop)	***O***(N)	***O***(K·L)	Each input byte triggers at most one hash-map lookup and one insert, both amortized ***O***(1). Space is dominated by K dictionary entries, each storing a string of average length L.
Code Width Mgmt	***O***(1) per code	***O***(1)	Simple integer comparison to decide when to widen the bit width.
Bit Packing (I/O)	***O***(N)	***O***(1)	Each emitted code is written bit-by-bit (9–16 bits); total bits proportional to N.
Decompression	***O***(N)	***O***(K·L)	Each code lookup is ***O***(1) via vector index; dictionary insertion is O(L) for string copy.
Total	***O***(N)	***O***(K·L)	LZW is linear in input size; space is bounded by the dictionary limit K times the average entry length L.

Note: the average entry length L is bounded by O(N/K) in the worst case (the dictionary fills with K entries whose total character content cannot exceed O(N)), so the total space is O(N + K) in practice. With K = 65,536, the dictionary overhead is a small constant relative to large inputs.

3.3.5 Discussion: Performance Boundaries & Limitations

1. Best-Case Scenario – Long files with highly repetitive multi-character patterns (e.g., DNA sequences like “ATCGATCGATCG...”, log files with repeating timestamps, or structured data formats with repeated field names). The dictionary quickly learns long substrings; after a warm-up phase, each emitted code (9–16 bits) can represent dozens or even hundreds of bytes. Compression ratios of 10x–20x are achievable — verified directly against the compiled

program: LZW achieved 19.96x compression (95.0% space savings) on a 10 KB file of repeating “AAAAABBBBB” patterns.

2. Worst-Case Scenario – Uniformly random data (encrypted files, already-compressed archives, or high-entropy byte sequences where no substring ever repeats). Every two-character combination is unique, so the dictionary fills with entries that are never reused. Each input byte is emitted as its own 9-bit code (worse than the original 8 bits), causing negative compression — the output file is approximately 12.5% larger than the input. This is a fundamental limitation of all dictionary-based methods on incompressible data.

3. Initial Dictionary Overhead – For very small files (fewer than 100 bytes), LZW's minimum code width of 9 bits per character (versus 8 bits in the original) means the compressed output is inherently larger before the dictionary has a chance to learn useful multi-character entries. This warm-up cost is amortized over longer inputs but is visible in micro-benchmarks. The group's own running example BABBACAC demonstrates this boundary directly: it compresses to exactly 8 bytes — equal to, not smaller than, the original — confirming that LZW breaks even rather than gains on an input this short (verified against the compiled program's actual output).

4. Variable-Width Code Strategy – This implementation uses variable-width codes that start at 9 bits and grow to 16 bits as the dictionary expands, which is more space-efficient than fixed 16-bit codes throughout (which would waste 7 bits per code on small dictionaries). The code width transitions at dictionary sizes of 512, 1024, 2048, ..., 32768. Once the dictionary reaches its limit of 65,536 entries, it freezes — no new entries are added, but compression continues using the existing entries. This freeze strategy (also used by Unix compress and GIF) avoids the complexity of dictionary-reset strategies while maintaining reasonable compression on long files.

5. Comparison with Other Algorithms

Property	RLE	Huffman	LZW
Compression Type	Run-length	Statistical (frequency)	Dictionary (substring)
Passes Required	1	2 (freq count + encode)	1 (adaptive)
Best On	Long runs of identical bytes	Skewed frequency distributions	Repeating multi-char substrings
Worst On	No consecutive repeats	Uniform distribution	Random / encrypted data
Metadata in File	None	Frequency table (header overhead)	None (self-contained)
Typical Use Case	Bitmaps, fax	JPEG, ZIP (as a stage)	GIF, TIFF, Unix compress

LZW's key advantage over Huffman is that it requires only a single pass and stores no metadata header — the decoder reconstructs the dictionary in lockstep with the encoder. Its key advantage over RLE is that it can exploit multi-character repetition patterns, not just runs of identical bytes. However, on data with extremely skewed single-character frequencies (e.g., a file that is 99% the letter “A”), Huffman may outperform LZW because it can assign a 1-bit code to the dominant character.

Section 4 (Experimental Results)

The table below reports measurements taken by running the group's compiled compressor directly against the Scenario 1 and Scenario 2 test files described in Section 2.2, using the tool's own reported Execution Time and Compression Ratio for each run.

File Size	Data Type	RLE	Huffman	LZW
10 KB	Standard English	1.53 ms (0.51x)	1.34 ms (1.84x)	5.90 ms (2.79x)
100 KB	Standard English	10.67 ms (0.51x)	10.80 ms (1.91x)	29.19 ms (3.11x)
1 MB	Standard English	103.01 ms (0.51x)	102.90 ms (1.92x)	257.87 ms (3.65x)
10 MB	Standard English	1051.91 ms (0.51x)	1095.00 ms (1.92x)	2275.21 ms (3.80x)
1 MB	Highly Repetitive	82.90 ms (2.50x)	61.13 ms (4.00x)	174.69 ms (108.80x)
1 MB	Random	93.38 ms (0.51x)	148.42 ms (1.34x)	290.22 ms (1.11x)

Growth Rate vs. Big-O Complexity

Execution time for all three algorithms scales roughly linearly with N as the file size increases 1000x, from 10 KB to 10 MB — consistent with the O(N)-dominated analysis in Section 3. LZW is consistently slower per byte than RLE or Huffman because of its extra hash-map lookup and insert on every character, but its growth rate still tracks linearly, confirming the average-case O(N) bound derived in Section 3.3.4.

Highest Compression Ratio

On standard English text, LZW achieves the best ratio (3.65x–3.80x), ahead of Huffman (around 1.92x) and RLE, which actually expands the file (0.51x). On the Highly Repetitive data, LZW pulls far ahead of the others (108.80x) because its dictionary can capture the entire repeating unit as a single reusable code, well beyond what Huffman's per-symbol entropy coding (4.00x) or RLE's per-run encoding (2.50x) can achieve.

Why: Tying Results to Theoretical Limits

RLE's failure on English text — and the resulting file expansion — directly confirms the worst-case boundary discussed in Section 3.1.5: English prose has almost no consecutive-byte repetition, so nearly every byte must be paired with its own count, roughly doubling the size. Huffman's plateau near 1.9x reflects it approaching the zeroth-order entropy of English letter frequencies; going further would require modeling multi-character context, which is exactly the gap LZW's dictionary approach fills. LZW's own weak point shows up on the Random data (1.11x), where there is no reusable structure to exploit, and most clearly on the group's own running example BABBACAC, where the 8-byte input is too short for the dictionary-building overhead to pay off at all (ratio exactly 1.00x, see Section 3.3.5).

Section 5 (References)

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[2] T. A. Welch, “A technique for high-performance data compression,” Computer, vol. 17, no. 6, pp. 8–19, Jun. 1984.

[3] J. Ziv and A. Lempel, “A universal algorithm for sequential data compression,” IEEE Trans. Inf. Theory, vol. 23, no. 3, pp. 337–343, May 1977.

Section 6 (Acknowledgment)

AI tools (Claude, Gemini) assisted with compiling the experimental benchmarks in Section 4 and the reference list in Section 5, using data generated directly by the group's own compiled program. All algorithm designs and analyses in this report as RLE, Huffman, LZW, and the bonus LZ77/Deflate sections — were written by the group.

Section 7 (Bonus Task: Advanced Compression - LZ77 & Deflate Algorithms)

7.1 Introduction & Application

- **Core Idea:** LZ77 is a dictionary-based lossless data compression algorithm using a sliding window strategy. The sliding window is partitioned into a **Search Buffer** (dictionary of previously processed data) and a **Lookahead Buffer** (upcoming data stream). It replaces repeating byte sequences with a tuple `(offset, length, nextChar)`, where `offset` indicates distance backward in the search window, `length` indicates the matched pattern length, and `nextChar` is the byte immediately following the match.

- **Real-world Applications:** LZ77 serves as the core matching engine in standard compression protocols including DEFLATE (ZIP, GZIP, PNG), LZMA (7-Zip), and GIF image formats.

7.2 Pseudocode

- **Compression**

```
Algorithm LZ77Compress(inputFile, outputFile):

    Input: Path to original file

    Output: Binary file containing (offset, length, nextChar) tokens

    buffer <- ReadAllBytes(inputFile)

    if buffer.IsEmpty() then:

        CreateEmptyFile(outputFile)

        return

    n <- buffer.Size()

    i <- 0

    windowSize <- 4095

    lookaheadSize <- 255

    writer <- OpenBinaryWriter(outputFile)

    while i < n do:

        bestMatchOffset <- 0

        bestMatchLength <- 0

        searchStart <- Max(0, i - windowSize)

        for j <- searchStart to i - 1 do:

            matchLen <- 0

              while (i + matchLen < n) and (buffer[j + matchLen] == buffer[i +
matchLen]) and (matchLen < lookaheadSize) do:
```

```
            matchLen <- matchLen + 1
        if matchLen > bestMatchLength then:
            bestMatchLength <- matchLen
            bestMatchOffset <- i - j
    if i + bestMatchLength < n then:
        nextChar <- buffer[i + bestMatchLength]
    else:
        nextChar <- 0
    writer.WriteToken(bestMatchOffset, bestMatchLength, nextChar)
    i <- i + bestMatchLength + 1
CloseFiles()
```

- **Decompression:**

```
Algorithm LZ77Decompress(inputFile, outputFile):
    Input: Compressed binary file containing tokens
    Output: Reconstructed original file
    reader <- OpenBinaryReader(inputFile)
    decompressedBuffer <- InitializeEmptyBuffer()
    while not reader.IsEOF() do:
        token <- reader.ReadToken()
        if token.length > 0 then:
            startPos <- decompressedBuffer.Size() - token.offset
            for k <- 0 to token.length - 1 do:
                byteVal <- decompressedBuffer[startPos + k]
                decompressedBuffer.Append(byteVal)
```

```
        decompressedBuffer.Append(token.nextChar)

    WriteBufferToFile(outputFile, decompressedBuffer)

    CloseFiles()
```

7.3 Step-by-Step Trace & Output Structure

- **Input String:** BABBACAC
- **Window Settings:** Search Window Size = 16 bytes, Lookahead Buffer Size = 16 bytes.

Step-by-Step Execution Trace:

Step	Search Buffer	Lookahead Stream	Longest Match Found	Token Emitted (Offset, Length, NextChar)	Updated Buffer
1	""	BABBACAC	None	(0, 0, 'B')	"B"
2	"B"	ABBACAC	None	(0, 0, 'A')	"BA"
3	"BA"	BBACAC	"B" at offset 2	(2, 1, 'B')	"BABB"
4	"BABB"	ACAC	"A" at offset 3	(3, 1, 'C')	"BABBAC"
5	"BABBAC"	AC	"AC" at offset 2	(2, 2, '\0')	"BABBACAC"

Encoded Stream: 5 Tokens emitted: (0,0,'B'), (0,0,'A'), (2,1,'B'), (3,1,'C'), (2,2,'\0').

Token Binary Size: Each token occupies 4 bytes (uint16_t offset, uint8_t length, uint8_t nextChar) equals 20 bytes total.

7.4 Complexity Analysis (Big-O)

Phase	Time Complexity	Space Complexity	Explanation
Search Matching	***O***(N . W_{search})	***O***(1)	For each byte position, scans up to W_{search} window bytes for substring matching.
Token Emission	***O***(N)	***O***(1)	Direct fixed-size structure write operation per token.
Decompression	***O***(N)	***O***(N)	Reconstructs data stream by direct index lookups in the output buffer.
Total Complexity	***O***(N . W_{search})	***O***(N)	Time linear in ***N*** given fixed search window size W_{search}.

7.5 Discussion: Performance Boundaries & Limitations

1. **Best-Case Scenario**
    - **Data Characteristic:** Large text files with heavily repeating sub-phrases across the sliding window range.
    - **Effect:** Long string patterns are compressed into compact 4-byte tokens, achieving high space savings percentages.
2. **Worst-Case Scenario**
    - **Data Characteristic:** Pseudo-random data or encrypted files without repeated sub-sequences.
    - **Effect:** Algorithm emits tokens with length 0 for every byte (0, 0, char), causing a file expansion from 1 byte to 4 bytes per character (400% size expansion).
3. **Window Size Trade-off**
    - Increasing Search Window size (W_{search}) improves compression ratio by locating distant matching patterns, but increases search time per byte. Fixed window sizes (e.g., 4095 bytes) bound execution time to ***O***(N).

7.6 The Deflate Algorithm: Combining LZ77 and Huffman

- **Core Idea:** Deflate is a two-stage scheme built directly on the two algorithms already implemented in this project — no new compression logic is introduced. The input is first compressed with `LZ77` (Section 7.1–7.4), producing a stream of `(offset, length, nextChar)` tokens. That token stream is then treated as an ordinary byte sequence and compressed a second time with `Huffman Coding` (Section 3.2), which assigns shorter codes to whichever token bytes recur most often. The two stages target different redundancy: LZ77 removes repeated substrings, and Huffman then removes the statistical skew left over in what remains — so the combination can compress further than either stage alone.

- **Real-world Applications:** This is exactly the design used by the real DEFLATE algorithm (RFC 1951), the compression method behind ZIP, GZIP, and PNG. Production DEFLATE additionally uses specialized length/distance alphabets and canonical Huffman codes, but the same “match, then entropy-code” two-stage structure is what is implemented here.

7.7 Pipeline (Pseudocode)

Because Deflate is a direct composition of two already-defined algorithms, its pseudocode is simply the pipeline that chains them — see 3.2.2 for HuffmanCompress/HuffmanDecompress and 7.2 for LZ77Compress/LZ77Decompress.

- **Compression:**

```
Algorithm DeflateCompress(inputFile, outputFile):

    tempFile ← CreateTemporaryFile()
```

```
    LZ77Compress(inputFile, tempFile)          // Section 7.2

    HuffmanCompress(tempFile, outputFile)      // Section 3.2.2

    DeleteFile(tempFile)
```

- **Decompression (stages run in reverse order):**

```
Algorithm DeflateDecompress(inputFile, outputFile):

    tempFile ← CreateTemporaryFile()

    HuffmanDecompress(inputFile, tempFile)

    LZ77Decompress(tempFile, outputFile)

    DeleteFile(tempFile)
```

7.8 Step-by-Step Trace on the Running Example

Running Deflate on the group's running example `BABBACAC`: Stage 1 (LZ77) produces the identical 5-token, 20-byte stream already traced in 7.3. Stage 2 (Huffman) then treats those 20 raw token bytes as its input alphabet and, per its own compiled behavior, writes a frequency-table header for that alphabet ahead of the entropy-coded payload (the same mechanism already verified for the tiny Huffman case in 3.2.3/3.2.5).

- **Result (verified against the compiled program's actual output):** 89 bytes — larger than both the 8-byte original and the 20-byte LZ77-only output. On an input this short, both stages' fixed overhead (LZ77's per-token structure, Huffman's frequency-table header) compounds rather than amortizes, which is expected and consistent with the small-file limitations already discussed for each stage individually (3.2.5, 7.5).

-

7.9 Experimental Comparison: LZ77 alone vs. Deflate

To see where the second stage actually pays off beyond the 8-byte running example, the group benchmarked Deflate against LZ77 alone on the same Scenario 1 and Scenario 2 files used in Section 4 (results below are the tool's own reported Compression Ratio and Space Savings for each run).

File Size / Data Type	LZ77 alone	Deflate (LZ77+Huffman)
10 KB — English	6,964 B (1.47x, 32.0%)	7,786 B (1.32x, 24.0%)
100 KB — English	65,016 B (1.57x, 36.5%)	53,504 B (1.91x, 47.8%)
1 MB — English	661,696 B (1.58x, 36.9%)	522,713 B (2.01x, 50.2%)
10 MB — English	6,612,052 B (1.59x, 36.9%)	5,201,737 B (2.02x, 50.4%)
1 MB — Repetitive	16,404 B (63.92x, 98.4%)	5,504 B (190.51x, 99.5%)
1 MB — Random	1,570,664 B (0.67x, -49.8%)	1,229,521 B (0.85x, -17.3%)

- **English text & Repetitive data:** for files of 100 KB or larger, adding the Huffman stage enables Deflate to significantly outperform standalone LZ77 (2.01x vs. 1.58x on 1 MB English text; 190.51x vs. 63.92x on 1 MB repetitive data). This is because Huffman efficiently compresses the small, low-entropy set of recurring tokens produced after LZ77 eliminates repetition.

- **Small-file crossover:** on the 10 KB English file Deflate is slightly worse than LZ77 alone (1.32x vs. 1.47x), the same header-overhead effect seen on the 8-byte running example, just far less extreme. The crossover point for this dataset falls between 10 KB and 100 KB.

- **Random data:** neither stage can remove real redundancy, but Deflate still expands less than LZ77 alone (0.85x / -17.3% vs. 0.67x / -49.8%): Huffman can still assign shorter codes to whichever LZ77 token pattern is most common (e.g. the fixed offset/length fields when almost every match fails and the token degrades toward `(0, 0, byte)`), partially offsetting LZ77's own expansion on incompressible input.

- **Execution-time cost:** the added Huffman stage is cheap. On the 10 MB English file, Deflate (54,243 ms) takes only about 449 ms longer than LZ77 alone (53,794 ms),

consistent with 3.2.4's O(N) bound for the Huffman stage being small next to LZ77's $O(N \cdot W_{search})$ search cost.

7.10 Complexity Analysis (Big-O)

Phase	Time	Space	Explanation
LZ77 Stage	$\boldsymbol{O}(N \cdot W_{search})$	$\boldsymbol{O}(1)$	As derived in 7.4; dominates total time for any non-trivial search window.
Huffman Stage	$\boldsymbol{O}(M + K' \log K')$	$\boldsymbol{O}(K')$	As derived in 3.2.4, applied to the LZ77-stage output of size M bytes with its own alphabet size $K' \leq 256$.
Total	$\boldsymbol{O}(N \cdot W_{search} + M \log K')$	$\boldsymbol{O}(K')$	Bounded in practice by the LZ77 stage, since $W_{search} \gg \log K'$ for the window sizes used here — matching the execution-time gap observed in 7.9.

7.11 Discussion: Performance Boundaries & Comparison

- **Best-Case Scenario**
  Data Characteristic: Large, highly repetitive files, as in the Repetitive row of 7.9.
  Effect: LZ77 does the heavy lifting on the repetition, and Huffman then squeezes further redundancy out of the resulting token stream — the two stages compound rather than compete.
- **Worst-Case Scenario**
  Data Characteristic: Small or already-random inputs.
  Effect: Both stages' fixed overhead (LZ77's per-token structure, Huffman's frequency-table header) accumulates without a matching compression gain — exactly what the running-example trace in 7.8 demonstrates (8 → 89 bytes).
- **Comparison with Standalone LZW**
  On the 1 MB English scenario, Deflate (2.01x) still trails this project's LZW (3.65x, Section 4) — mainly because the group's LZ77 stage uses a comparatively small, fixed 4095-byte search window (7.2) with a brute-force search (7.4), while LZW's dictionary can reference matches anywhere earlier in the file. A larger search window or a hash-chain search, as used by production DEFLATE, would likely close some of this gap, at the cost of the $O(N \cdot W_{search})$ search time already identified as LZ77's bottleneck in 7.9.